

Automotive Demand Response Analyzer Using DuckDB

DSCI 551 Final Report

May Tong

Introduction & Motivation

The goal of this project was to explore the internal architecture of a modern analytical database system and understand how its internal design choices affect application behavior and query performance. For this project, I chose DuckDB and built an analytical application called the **Automotive Demand Response Analyzer**.

The idea for this project was inspired by my previous internship experience in automotive consulting, where regional sales performance and time components were important parts of business analysis. During that internship, I saw how large amounts of sales were used to evaluate demand trends and dealership behavior, as well as to compare regional performance. I wanted to build a simplified and slightly different analytical system inspired by that type of workflow while also connecting it to database internals.

I chose DuckDB because it is designed specifically for analytical workloads. Unlike traditional row-based systems such as MySQL, DuckDB uses a columnar storage model and vectorized query execution engine. These architectural decisions make DuckDB particularly efficient for aggregation-heavy workloads such as group-by operations, time-series analysis, and large table scans.

The main focus areas of this project are:

- Columnar storage
- Vectorized query execution

The application demonstrates how these internals improve analytical query performance and reduce unnecessary data access.

Database System Overview

DuckDB is an open-source analytical database system designed for Online Analytical Processing (OLAP) workloads. It is often described as an “in-process analytical database” because it runs directly inside applications such as Python notebooks without requiring a separate database server.

DuckDB is optimized for analytical queries involving:

- Large scans
- Aggregations
- Filtering

- Group-by operations
- Time-series analysis

Unlike transactional systems such as MySQL or PostgreSQL, DuckDB is not primarily designed for high-concurrency transaction processing. Instead, it is optimized for fast analytical workloads over large datasets.

Some of the main features of DuckDB include:

- Columnar storage architecture
- Vectorized query execution
- SQL support
- Tight integration with Python and Pandas
- Lightweight deployment with no server setup

DuckDB is commonly used for local analytics workflows, data science tasks, and exploratory analysis. Its design philosophy is similar to analytical systems such as Spark, Snowflake, BigQuery, and Redshift, where the focus is on efficiently scanning and aggregating large datasets.

One major advantage of DuckDB is that it combines the simplicity of SQLite with the analytical performance characteristics of a data warehouse engine.

Internal Architecture

Columnar Storage

One of the most important aspects of DuckDB's architecture is its columnar storage model. In traditional row-based databases, data is stored row-by-row. For example:

name	age	salary
Alice	25	70000
Bob	30	90000

DuckDB instead stores data column-by-column:

Names:

- Alice
- Bob

Ages:

- 25
- 30

Salaries:

- 70000
- 90000

This design is extremely important for analytical workloads because analytical queries often use only a subset of columns. For example:

```
SELECT region, SUM(units_sold)
FROM sales
GROUP BY region;
```

This query only requires the `region` and `units_sold` columns. DuckDB therefore scans only those columns instead of reading the entire row. This reduces:

- Disk I/O
- Memory usage
- CPU overhead

As datasets become larger, the benefits of columnar storage become more significant.

Vectorized Query Execution

DuckDB also uses vectorized query execution. Instead of processing data one row at a time, DuckDB processes data in batches called vectors. A vector may contain thousands of values. For example, instead of computing:

```
SUM(row1)
SUM(row2)
SUM(row3)
```

DuckDB processes a batch of values together. This improves performance because:

- CPU instructions can operate on batches efficiently
- Memory access becomes more cache-friendly
- Function call overhead is reduced

Query Execution Workflow

The general execution flow for a query in DuckDB is:

1. Parse SQL query
2. Generate execution plan
3. Perform columnar scans
4. Execute vectorized operators
5. Aggregate and return results

For example, the query:

```
SELECT region, SUM(units_sold)
FROM sales
GROUP BY region;
```

produces an execution plan containing:

- Sequential scan
- Projection
- Hash group by
- Aggregation

Application Design

Application Overview

The application developed for this project is called the **Automotive Demand Response Analyzer**. It aims to analyze how incentives and pricing relate to vehicle sales across regions and time. The application is designed to simulate real-world automotive sales analysis workflows involving regional sales analysis, incentive analysis, demand trends, and time-series aggregation.

Dataset Design

The dataset used in this project was synthetically generated using Python. The dataset includes the following fields:

Column	Description
sale_date	Timestamp of sale
region	Geographic sales region
segment	Vehicle category
msrp	Vehicle price
incentive	Discount/incentive amount
units_sold	Units sold
revenue	Calculated revenue

The synthetic data was intentionally designed to behave more realistically instead of being completely random. For example, different regions have different average incentive levels, different vehicle segments have different demand patterns, and incentives partially influence units sold. This allowed the analytical queries to produce more realistic and meaningful patterns. The final dataset contains 300,000 rows.

Application Workflow

The workflow of the application is:

1. Generate synthetic automotive sales data
2. Load data into DuckDB

3. Execute analytical SQL queries
4. Visualize aggregated results
5. Analyze execution behavior using EXPLAIN

Mapping Internals to Application Behavior

Feature 1: Regional Demand Analysis

```
SELECT region, SUM(units_sold)
FROM sales
GROUP BY region;
```

What the Application Does: This query calculates total vehicle sales for each region.

What DuckDB Does Internally: DuckDB scans only the `region` and `units_sold` columns because of its columnar storage architecture. The aggregation is then executed using vectorized processing.

Why This Matters: DuckDB avoids scanning unnecessary columns and performs aggregation efficiently in batches.

Feature 2: Monthly Sales Trends

```
SELECT DATE_TRUNC('month', sale_date),
       SUM(units_sold)
FROM sales
GROUP BY 1;
```

What the Application Does: This query computes monthly sales trends.

What DuckDB Does Internally: DuckDB scans only `sale_date` and `units_sold`. The `DATE_TRUNC` operation and aggregation are applied using vectorized execution.

Why This Matters: This allows efficient time-series analysis without scanning full rows or processing data row-by-row.

Feature 3: Incentive vs Demand Analysis

```
SELECT region,
       AVG(incentive),
       SUM(units_sold)
FROM sales
GROUP BY region;
```

What the Application Does: This query compares average incentives and total sales across regions.

What DuckDB Does Internally: DuckDB scans only `region`, `incentive`, and `units_sold`. The database performs multiple aggregations simultaneously using vectorized operators.

Why This Matters: This demonstrates how DuckDB efficiently handles multi-metric, aggregation-heavy analytical workloads while minimizing unnecessary data access.

Comparison with MySQL and MongoDB

MySQL is a traditional relational database system optimized for transactional workloads (OLTP). It uses row-based storage, which works well for frequent inserts, updates, concurrent users, and transaction-heavy systems. However, row-based storage is less efficient for analytical workloads. For example, in MySQL, the database often reads full rows even though only two columns are needed, whereas DuckDB instead scans only the required columns.

MongoDB is a NoSQL document database. Instead of storing structured rows and columns, MongoDB stores flexible JSON-like documents. However, MongoDB is not specifically optimized for analytical aggregation workloads in the same way as DuckDB.

Strengths of DuckDB:

- Faster analytical aggregation
- Better column scanning efficiency
- Better suited for OLAP workloads
- More efficient for large analytical queries
- Appropriate use cases include data analysis, local analytics, data science workflows, and aggregation-heavy workloads

Strengths of MySQL:

- Better transactional support
- Better concurrency handling
- Better for web applications
- Mature ecosystem for production systems
- Appropriate use cases include web applications, transaction systems, and multi-user production systems

Strengths of MongoDB:

- Flexible schema design
- Easier handling of nested data
- Better horizontal scaling support
- Better for rapidly changing application data and semi-structured data
- Appropriate use cases include flexible application data, content management, rapid prototyping, and semi-structured datasets

Limitations & Lessons Learned

Limitations

One limitation of this project is that the dataset is synthetic rather than real-world production data. Although the dataset was designed to behave realistically, it still cannot fully capture the complexity of real automotive sales systems. An easy fix is acquiring a real-world dataset with similar information and applying the application to it.

Another limitation is that the application focuses primarily on analytical queries rather than transactional workloads.

Additionally, the project focuses mainly on columnar storage and vectorized execution, and does not deeply explore distributed systems, replication, and advanced indexing structures.

Finally, DuckDB itself is not designed for highly concurrent multi-user transactional systems, so it would not be able to replace databases such as MySQL for many production applications.

Challenges

Several challenges were encountered during the project:

- Understanding vectorized execution required additional research
- Interpreting the execution plan was initially difficult
- Designing realistic synthetic data that balanced realism and simplicity
- Explaining database internals clearly while connecting them to application behavior

Lessons Learned

This project helped deepen my understanding of:

- Columnar database systems
- Query execution planning
- Vectorized execution
- Analytical database workloads
- Differences between OLTP and OLAP systems

One major lesson learned is that databases can execute the same SQL query very differently internally. This project also showed how strongly storage layout affects performance.

Future Improvements

Possible future improvements include:

- Using real automotive datasets
- Adding more advanced analytical features
- Benchmarking against MySQL or PostgreSQL directly
- Exploring distributed analytical systems
- Adding interactive dashboard functionality

Project Links

GitHub Repository Link:

<https://github.com/MayoooooT/duckdb-demand-response-analyzer>

The repository contains:

- Source code (.ipynb notebook)
- README file
 - Dataset information
- requirements.txt